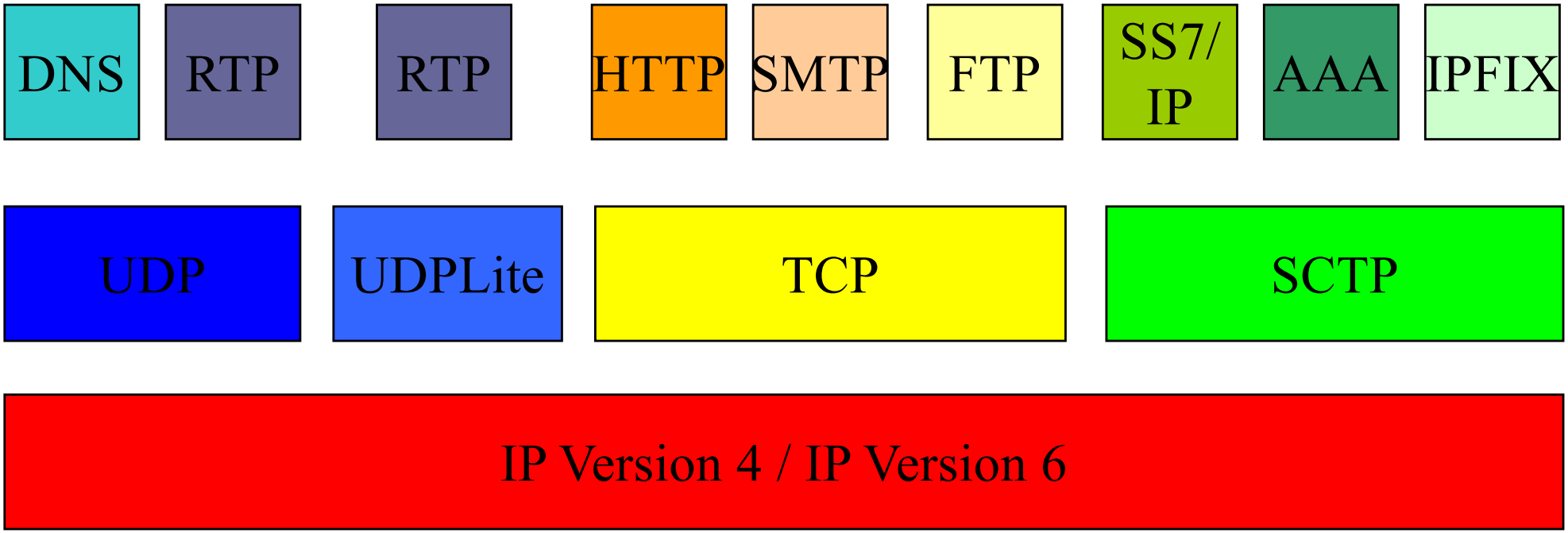
Netzwerkprogrammierung

Vorlesung 3

Die Internet Protocol (IP) Protokollfamilie



Probleme bei der Programmierung

- socklen_t nicht verfügbar.
- sin_len nicht immer vorhanden.
- Anzahl der Filedescriptoren sehr unterschiedlich.
- Unterschiedliche Headerfiles.
- Unterschiedliche Bibliotheken.

Handhabung von Typen

- Das configure Script testet, ob die entsprechenden Typen vorhanden sind. Falls nicht, werden Ersatztypen definiert.
- AC_CHECK_TYPE(ssize_t)
- AC_CHECK_TYPE(socklen_t) funktioniert nicht, da nicht in sys/types.h definiert

Handhabung von Strukturkomponenten

- Hierzu compiliert man ein Testprogramm. Wenn es eine benutzte Komponente nicht gibt, erzeugt der Compiler einen Fehler.
- AC_TRY_COMPILE(incl, body, ok, else)
  - Es wird versucht, dass Programm mit den includes incl und dem Body body zu übersetzen. Im Erfolgsfall werden die Kommandos ok ausgeführt, sonst else.
- AC_DEFINE(identifier)
  - identifier wird definiert
- Anwendung: siehe Beispiel

Handhabung von Funktionen

- AC_CHECK_FUNCS(func, ok, else)
  - ‚HAVE_FUNC‘ ist gegebenenfalls definiert, die ok Kommandos ausgeführt.
  - sonst else Kommandos ausgeführt.
- AC_CHECK_LIB(lib, func)
  - Prüft, ob func in lib definiert ist und fügt lib gegenenfalls zu den zu benutzenden Libraries hinzu.
- Sollte auch kombiniert werden, siehe Beispiel.

Prüfen der Umgebung

- AC_CANONICAL_HOST
- cpu-manufacturer-os triplet
  - Beispiel: powerpc-apple-darwin6.5
- case $host_os in
  solaris*)
  CFLAGS="$CFLAGS -DSOLARIS“
  ;;
  esac
- Im Code dann mit #ifdef SOLARIS arbeiten

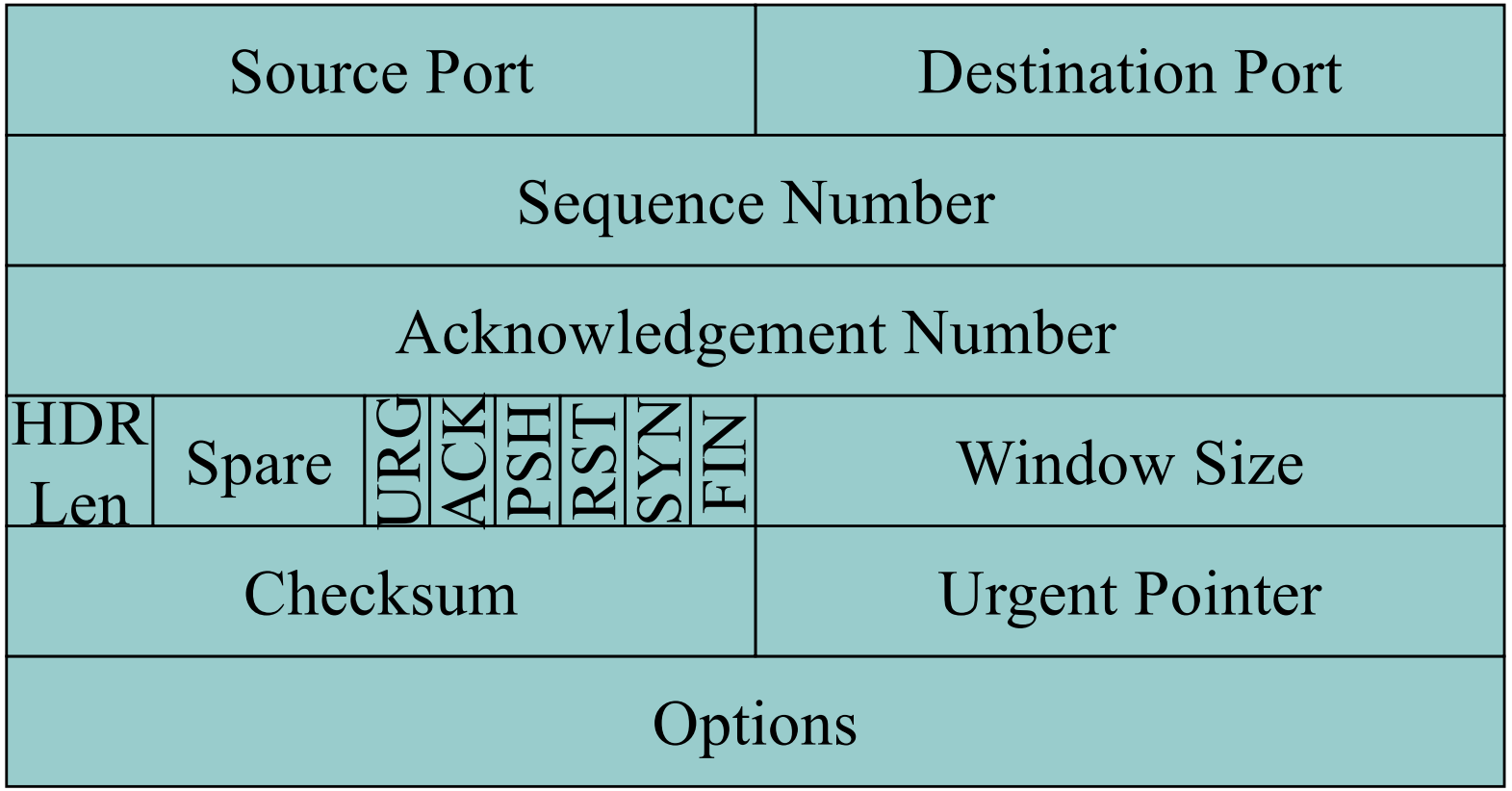
Beispiel: udp_examples

- Der Source wurde so modifiziert, dass er unter Linux, Solaris, FreeBSD und MacOSX läuft.
- Unterschied: Entwickler <-> User.
- Erstellt werden müssen bootstrap, Makefile.am und configure.in.

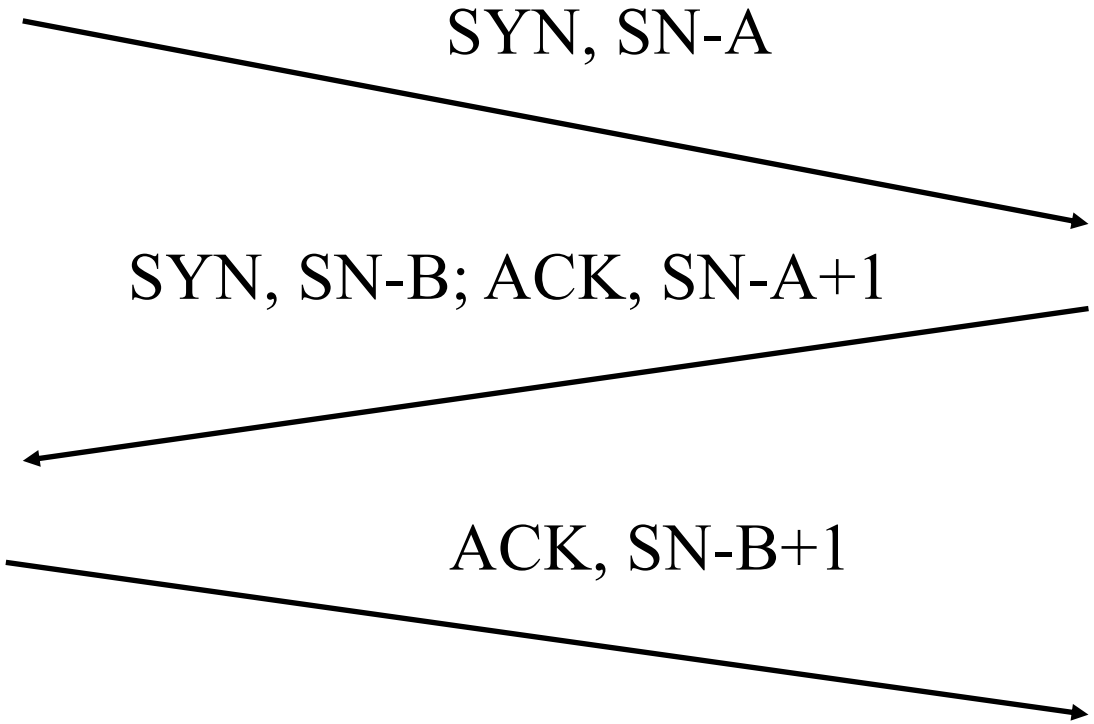
Transmission Control Protocol

- Bytestreamorientiert
- Gesichert
- Flowcontrol
- Congestioncontrol
- Segmentierung
- Singlehoming
- Unicast

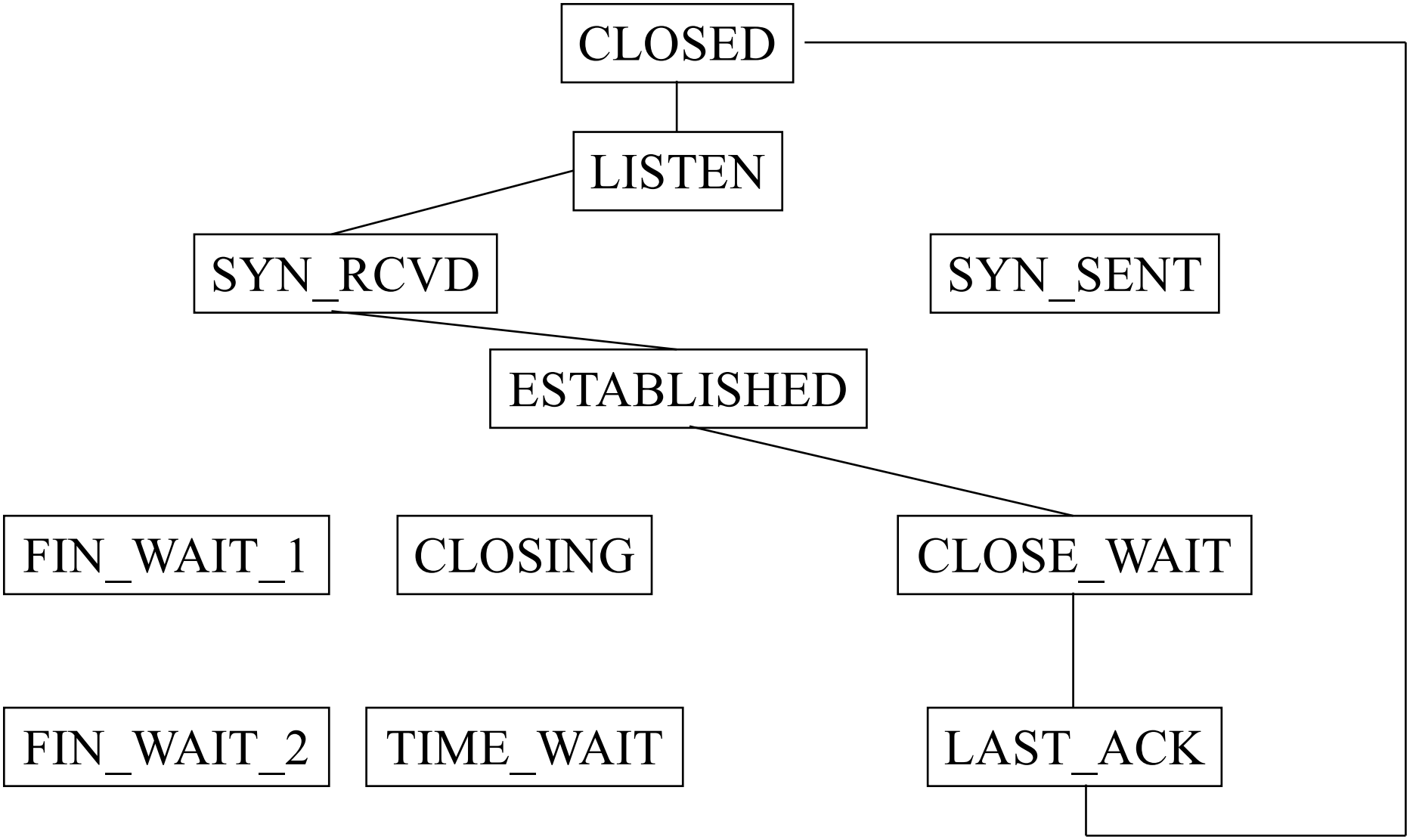
TCP Header


Source Port
Destination Port
Sequence Number
Acknowledgement Number
HDR Len
Spare
URG
ACK
PSH
RST
SYN
FIN
Window Size
Checksum
Urgent Pointer
Options

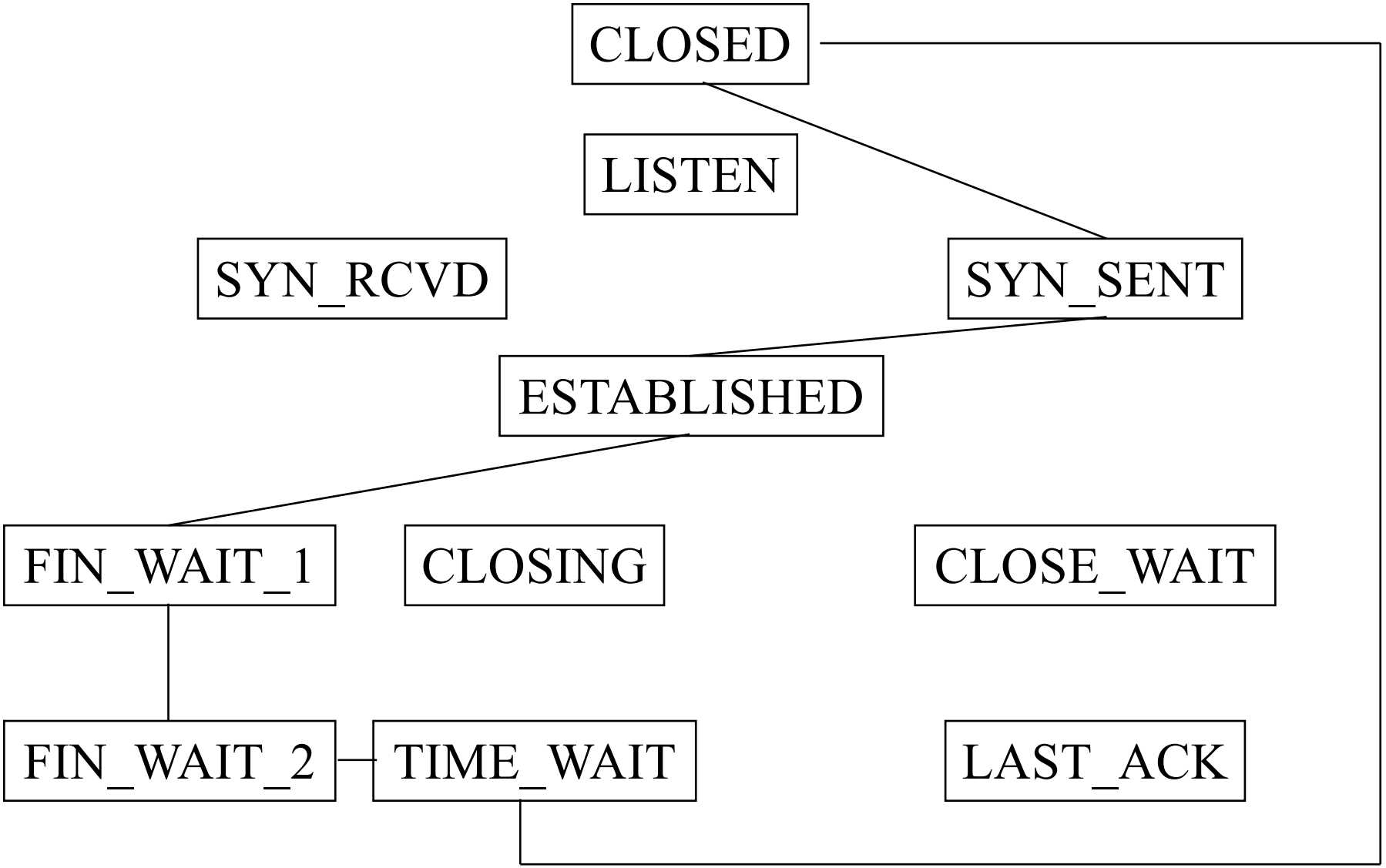
Der Verbindungsaufbau



TCP State Engine (Server)



TCP State Engine (Client)



Echo Protocol für TCP

- RFC 862
- Well known Port: 7
- Abbau der TCP Verbindung von Client initiert.

Discard Protocol für TCP

- RFC 863
- Well known Port: 9
- Abbau der TCP Verbindung von Client initiert.

Daytime Protocol für TCP

- RFC 867
- Well known Port: 13
- Sende auf jeder akzeptierten Verbindung die aktuellen Zeit zurück und stoße den Verbindungsabbau an.

Chargen Protocol für TCP

- RFC 864
- Well known Port: 19
- Abbau der TCP Verbindung von Client initiert.

listen call

- int listen(int fd, int backlog);
  - fd: Filedecriptor
  - backlog: Anzahl der maximal zu puffernden Verbindungsanforderungen.
- Gibt 0 zurück, wenn alles geklappt hat, sonst -1.

accept call

- int accept(int fd,
  struct sockaddr* addr, socklen_t *len)
  - fd: Filedescriptor.
  - addr: Pointer auf Addressstruktur
  - len: Pointer auf Länge. Value-Result Parameter
- Gibt einen Filedescriptor für die akzeptierte Verbindung zurück oder -1 im Fehlerfall.

connect call

- int connect(int fd,
  const struct sockaddr *addr,
  socken_t len)
  - fd: Filedescriptor;
  - addr: Zeiger auf Adresse des Servers;
  - len: Länger der Adresse.
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

close call

- int close(int fd);
  - fd: Filedescriptor
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

shutdown call

- int shutdown(int fd, int how);
- how ist SHUT_RD, SHUT_WR, SHUT_RDWR.
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

Discard Server

- Well known Port: 9.
  - socket()
  - bind()
  - listen()
  - accept()
  - recv()
  - close()

Ein Discard Server (vereinfacht)

```
fd = Socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
memset((void *) &server_addr, 0, sizeof(server_addr));
server_addr.sin_family      = AF_INET;
server_addr.sin_len         = sizeof(struct sockaddr_in);
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
server_addr.sin_port        = htons(DISCARD_PORT);
Bind(fd, (const struct sockaddr *) &server_addr, sizeof(server_addr))
Listen(fd, 1);
while (1) {
        memset((void *) &client_addr, 0, sizeof(client_addr));
        client_addr_len = (socklen_t) sizeof(client_addr);
        cfd = Accept(fd, (struct sockaddr *) &client_addr,
                         &client_addr_len)
        do {
                len = Recv(cfd, (void *) buf, sizeof(buf), 0);
        } while (len > 0)
        Close(cfd);
}
Close(fd);
return(0);
```

Problem beim Server

- Bricht man einen Server mit Control-C ab, wenn er noch eine Verbindung mit einem Client hat, so kann man ihn nicht sofort wieder starten.
- Der socket befindet sich im State TimeWait.
- Der bind Aufruf gibt einen Fehler zurück.

inet_pton

- int inet_pton(int family,
  const char *strptr,
  void *addrptr)
- family: AF_INET oder AF_INET6
- strptr: Adresse als String
- addrptr: Adresse in binärer Form als struct in_addr oder struct inaddr_6
- Gibt bei Erfolg 1, sonst 0.

inet_ntop

- const char inet_ntop(int family, const void *addrptr,
  char * strptr, size_t len)
- family: AF_INET oder AF_INET6
- addrptr: Adresse in binärer Form als struct in_addr oder struct in6_addr.
- strptr, len: Speicherbereich der Länge len, in dem die Adresse als String gespeichert wird.
- Gibt bei Erfolg strptr zurück, sonst NULL.